// TRAINING

Lesson 1 - Working with Akka.NET and Akka.Cluster

Welcome to the first stage of “End to End Akka.NET Distributed Programming!”

In this first lesson, we’re going to learn how to connect multiple Akka.NET `ActorSystem` s together using Akka.Remote and Akka.Cluster to form cooperative, elastic, highly scalable applications.

Akka.CQRS Sample Overview

Akka.CQRS is a reference architecture for Akka.NET, intended to illustrate the following Akka.NET techniques and principles:

1. Command-Query Responsibility Segregation - the Akka.NET actors who consume write events use distinctly different interfaces from those who consume read events
2. Akka.Cluster - a module that allows Akka.NET developers to create horizontally scalable, peer-to-peer, fault-tolerant, and elastic networks of Akka.NET actors.
3. Akka.Cluster.Sharding - a fault-tolerant, distributed tool for maintaining a single source of truth for all domain entities.
4. Akka.Persistence - a database-agnostic event-sourcing engine Akka.NET actors can use to persist and recover their data, thereby making it possible to move a persistent entity actor from one node in the cluster to another.
5. Akka.Cluster.Tools - this sample makes use of `DistributedPubSub` for publishing events across the different nodes in the cluster and `ClusterSingleton` , to ensure that all read-side entities are up and running at all times.
6. Petabridge.Cmd - a command-line interface for Akka.NET that we use for watching multiple nodes in the cluster and inspecting their operations.
7. Akka.Bootstrap.Docker - this sample uses Docker and `docker-compose` to run the sample, and Akka.Bootstrap.Docker is used to inject runtime environment variables

into the Akka.NET HOCON configuration at run-time.

Throughout this course, we will gradually implement all of these tools alongside Docker and Kubernetes.

Trading Services

The write-side of the CQRS operation, the Trading Services, are primarily interested in the placement and matching of new trade orders for buying and selling of specific stocks.

The Trading Services are driven primarily through the use of three actor types:

1. `BidderActor` - runs inside the "Trade Placement" services and randomly bids on a specific stock;
2. `AskerActor` - runs inside the "Trade Placement" services and randomly asks (sells) a specific stock; and
3. `OrderBookActor` - the most important actor in this scenario, it is hosted on the "Trade Processor" service and it's responsible for matching bids with asks, and when it does it publishes `Match` and `Fill` events across the cluster using `DistributedPubSub`. This is how the `AskerActor` and the `BidderActor` involved in making the trade are notified that their trades have been settled. All events received and produced by the `OrderBookActor` are persisted using Akka.Persistence.MongoDb.

The domain design is relatively simple otherwise and we'd encourage you to look at the code directly for more details about how it all works.

Pricing Services Domain

The read-side of the cluster, the Pricing Services consume the `Match` events for specific ticker symbols produced by the `OrderBookActor`s inside the Trading Services by receiving the events when they're published via `DistributedPubSub`.

The `MatchAggregator` actors hosted inside Akka.Cluster.Sharding on the Pricing Services nodes are the ones who actually consume `Match` events and aggregate the `Match.SettlementPrice` and `Match.Quantity` to produce an estimated, weighted moving average of both volume and price.

> *We won't be using the Pricing Services until lesson 4, when we begin to work with Akka.Cluster.Tools and Akka.Cluster.Sharding.*

Getting Started

To start this lesson, we need to checkout to the `start` branch in our git repository:

```
PS> git checkout start
```

At the moment, we have two services that are designed to work together:

- Akka.CQRS.TradePlacers.Service - a service that creates `Bid` and `Ask` events against specific stocks and waits for `Fill` events that indicate that open orders have been filled.
- Akka.CQRS.TradeProcessor.Service - a service that attempts to match outstanding `Bid` orders with `Ask` orders for the same stock symbol, which in turn produces `Fill` and `Match` events.

However, these services have no way of communicating - this is what we intend to fix through the use of Akka.Remote and Akka.Cluster.

Inter-process Communication with Akka.Remote

When most developers start learning Akka.NET they start with actors that communicate solely through in-memory message passing. The asynchronous, fire-and-forget messaging syntax we all learned over and over again in Akka.NET Bootcamp:

```
private readonly IActorRef _tradeGateway;

Receive<DoAsk>(_ ⇒
{
    var ask = CreateAsk();
    _asks[ask.OrderId] = ask;
    _tradeGateway.Tell(new ConfirmableMessage<Ask>(ask,
_confirmationId++, _traderId));
    _log.Info("ASK ${0} for {1} units of {2}", ask.AskPrice,
```

```
ask.AskQuantity, _tickerSymbol);
});
```

One of the most powerful features of Akka.NET is its ability to take this in-memory message passing model and transparently translate it into a *network message passing model* using just a few lines of configuration - this is precisely what Akka.Remote does.

After installing the Akka.Remote NuGet package into your Akka.NET application, your `ActorSystem` can begin receiving inbound TCP connections from other `ActorSystem` s running Akka.Remote using the following HOCON:

```
akka.actor.provider = remote
akka.remote.dot-netty.tcp{
        hostname = localhost
        port = 9001
}
```

This will create an inbound listening address that looks like the following:



In Akka.Remote we're able to use network transports, typically TCP sockets, to allow multiple `ActorSystem` s to communicate with each other over the network. If you had a top-level actor running with the name "foo" inside your "MySystem" `ActorSystem` , another Akka.NET actor running in a different process could send it a message with the following `ActorSelection` :

```
public class MyActor : ReceiveActor{
        public MyActor(){
                ReceiveAny(_ ⇒ Console.WriteLine(_.ToString()));
        }
```

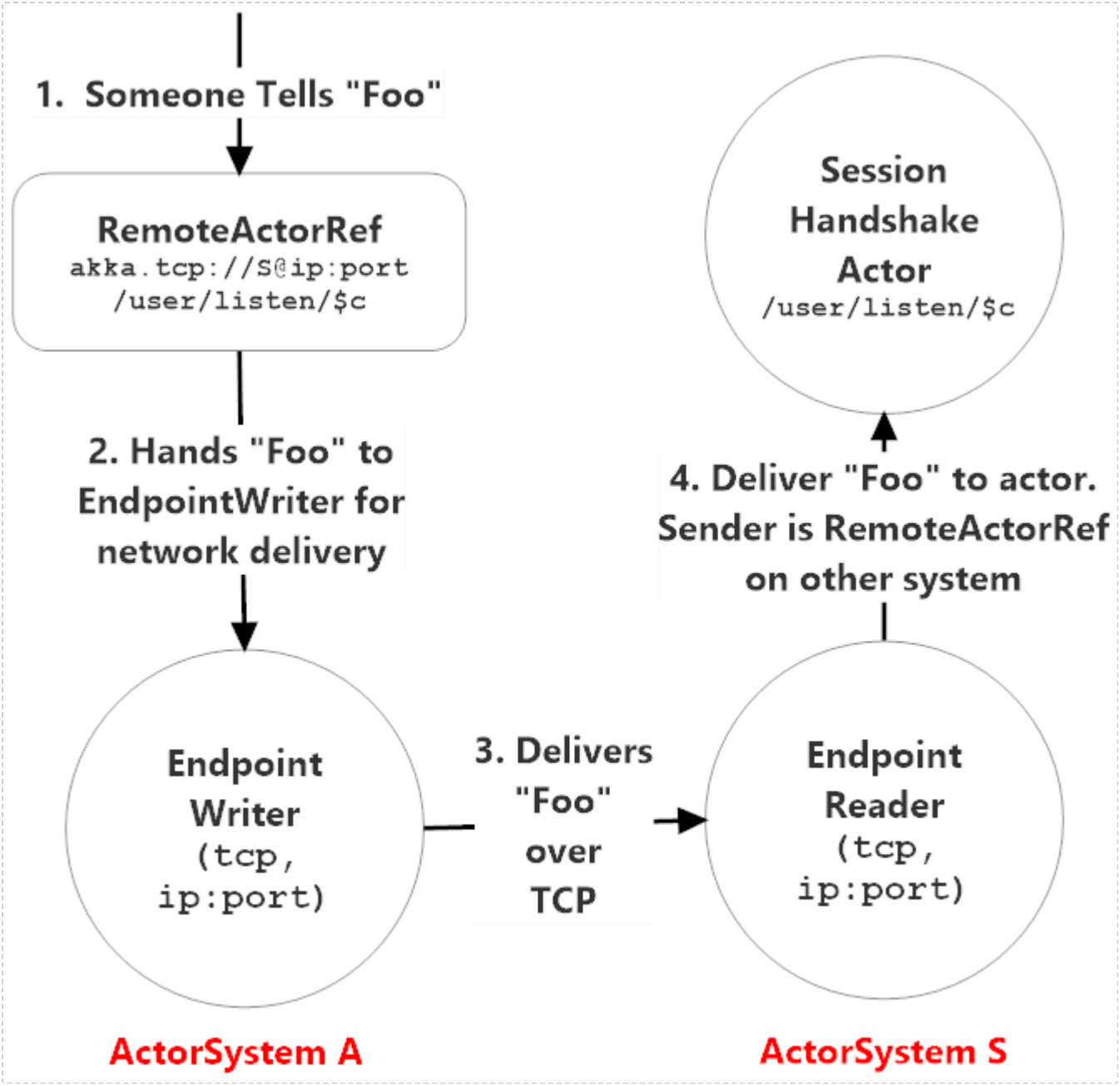
```
        protected override void PreStart(){
                var myRemoteActorSelection =
Context.ActorSelection("akka.tcp://MySystem@localhost:9001/user/f
oo");
                myRemoteActorSelection.Tell("hi");
        }
}
```

The “hi” message will be serialized into a byte array, transmitted over TCP, received by our local TCP endpoint, deserialized back into its string representation, and then successfully delivered to our `/user/foo` actor as though it were a local message.

What’s really powerful though is what would happen if we were to reply *back* to the actor who sent us the “hi” message:

```
public class FooActor : ReceiveActor{
        public FooActor(){
                Receive<string>(s ⇒ Sender.Tell(s));
        }
}
```

When we reply back to the `Sender` of the “hi” message, we’re actually replying back to a `RemoteActorRef` - a special actor reference that points back to an actor that is running in a different `ActorSystem` than the one our current actor is running inside of! This is where the real power of Akka.Remote comes into play: `RemoteActorRef` s are location transparent - we don’t even need to know that we’re talking to an actor on the other side of the network when we send it a message because *remote message passing and local message passing don’t require any code changes on the part of the Akka.NET developer*.



That all is this works, mostly transparently, makes it extremely easy for Akka.NET developers to build networked systems without large amounts of plumbing.

But how can we ensure that those networks are built with a tendency towards high availability and fault tolerance? We do that through the adoption of Akka.Cluster: another Akka.NET component that uses Akka.Remote under to construct resilient, scalable networks of Akka.NET processes.

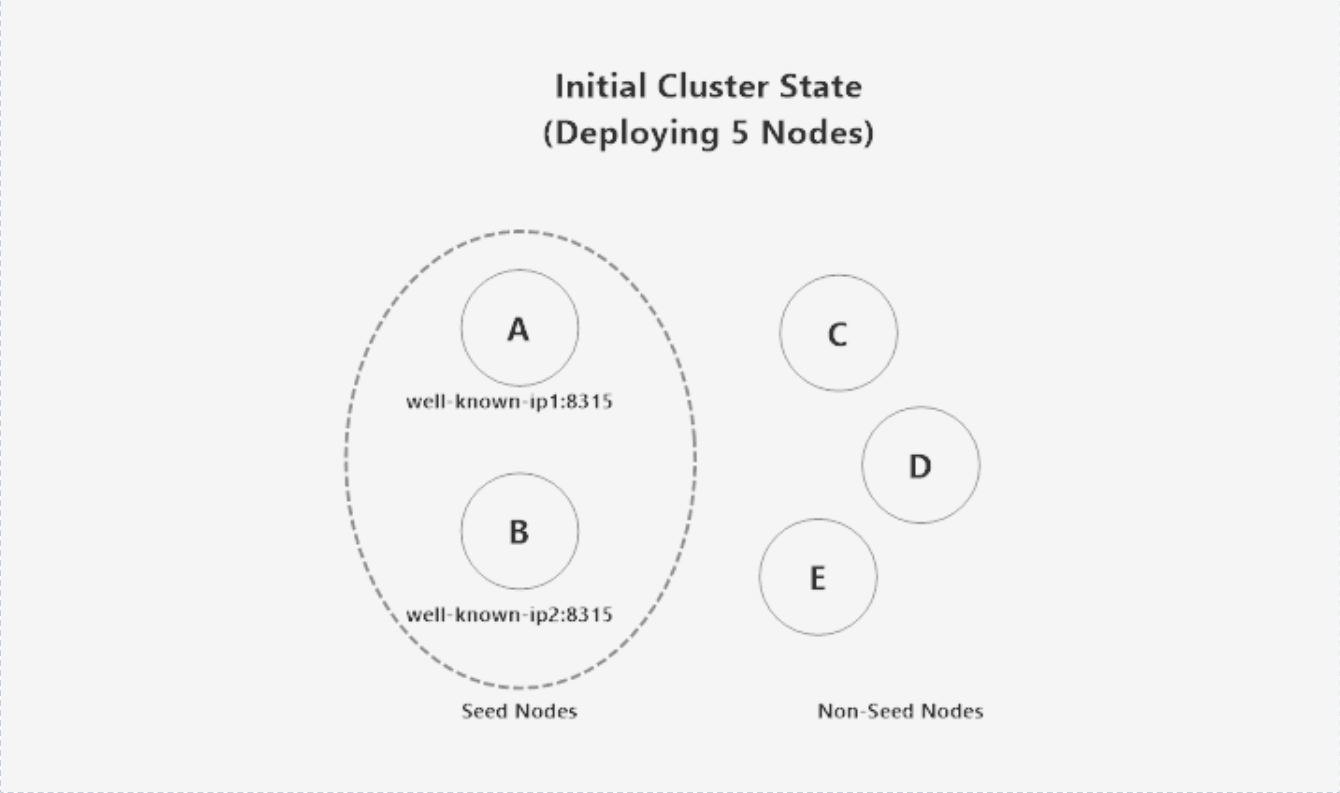
Building Highly Available Applications with Akka.Cluster

A cluster represents a fault-tolerant, elastic, decentralized peer-to-peer network of Akka.NET applications with no single point of failure or bottleneck. The Akka.Cluster gives us the ability to create these types of applications.

Akka.Cluster depends on Akka.Remote, but has a similarly simple configuration for most use cases:

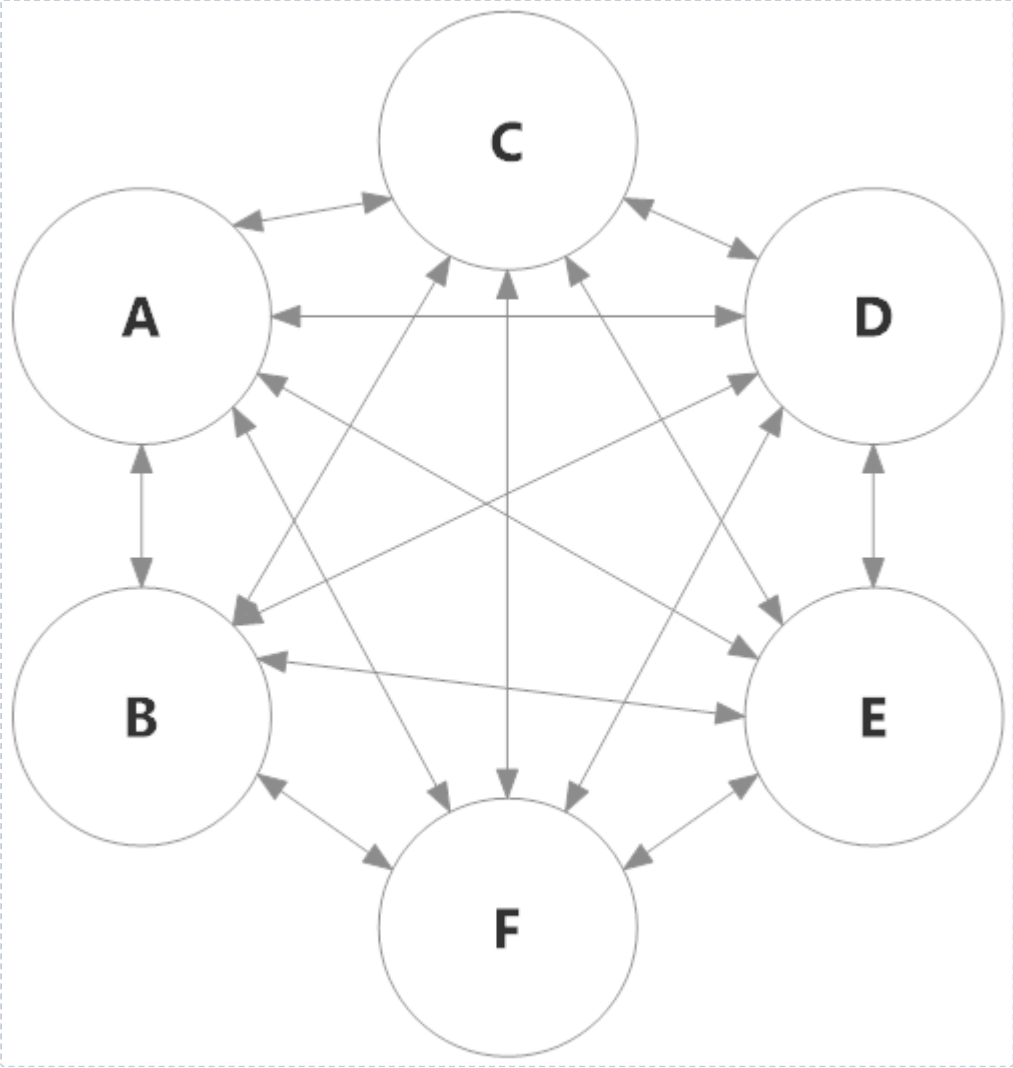
```
akka {
    actor.provider = cluster
    remote {
        dot-netty.tcp {
            port = 8081
            hostname = localhost
        }
    }
    cluster {
        seed-nodes = ["akka.tcp://ClusterSystem@localhost:8081"]
    }
}
```

The way Akka.Cluster works is thus: at startup each node will attempt to connect to the `ActorSystem` s running at the addresses specified inside the `akka.cluster.seed-nodes` collection.



Once a node is able to successfully join at least one of the seed nodes, it'll be introduced to the other nodes in the cluster. Eventually, every node will have a

connection to every other node in the cluster, like so:



Every node has a connection to every other node - this allows for easy communication between nodes, rapid failure detection, and quick propagation of changes in the cluster's topology, such as when a node joins or leaves the cluster.

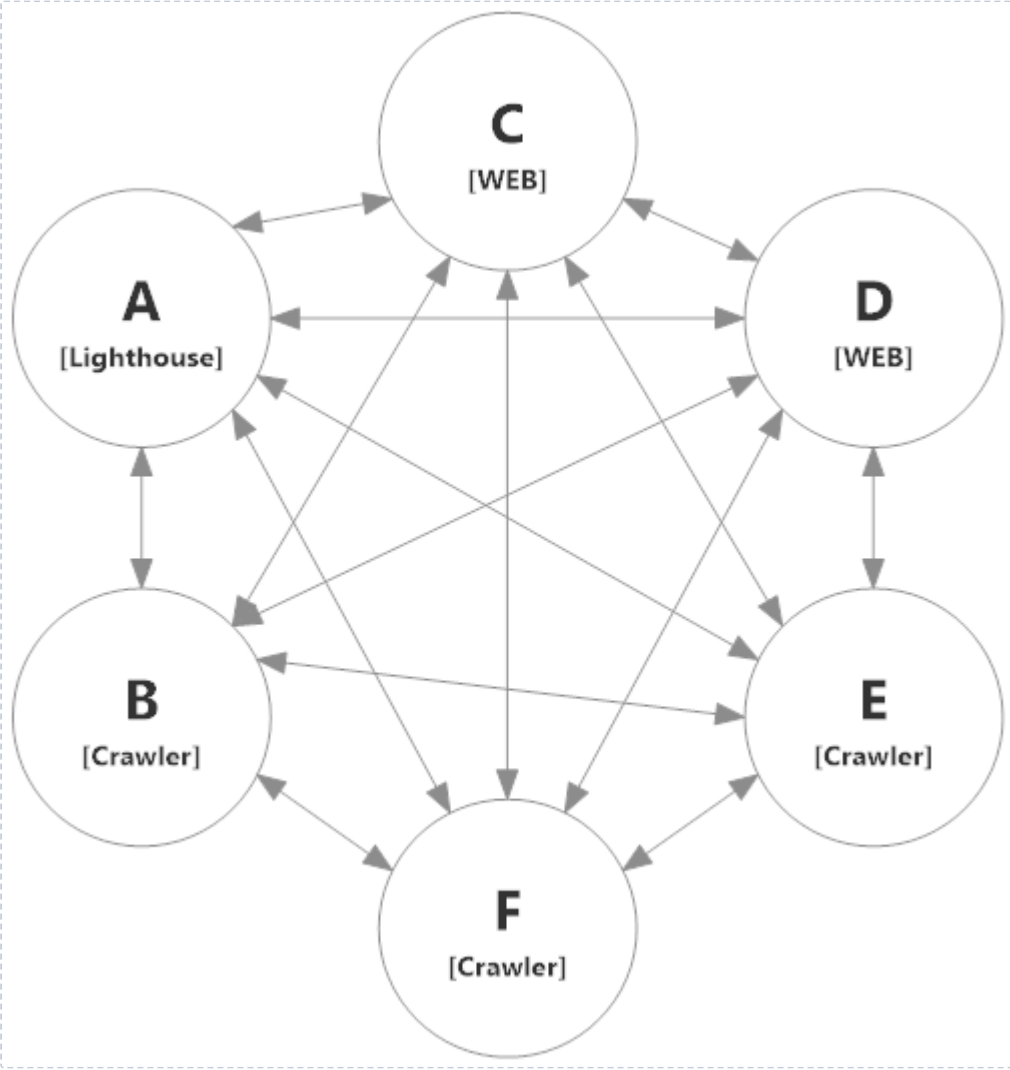
Cluster Roles

What makes Akka.NET clusters particularly useful, however, is the fact that we can describe what capabilities are of each node in the cluster through the "roles" declaration inside the `akka.cluster` HOCON:

```
akka {
    actor.provider = cluster
    remote {
        dot-netty.tcp {
            port = 8081
            hostname = localhost
```

```
            }
        }
        cluster {
            seed-nodes = ["akka.tcp://ClusterSystem@localhost:8081"]
                  roles = ["crawler"]
        }
    }
```

Roles are used to help distinguish different Akka.NET applications and services that are all working together, cooperatively, inside the same Akka.NET cluster.



Roles are essentially just tags that can be used to tell other Akka.Cluster services where to send traffic. Akka.Cluster.Sharding, DistributedPubSub, ClusterClient, and the router actors built-in to Akka.Cluster can all utilize roles to direct their message traffic towards specific areas of our Akka.NET cluster. We'll be using them heavily throughout this workshop.

Distributing State and Messages Using Roles and Clustered Routers

In this first lesson, we're going to enable our Akka.CQRS.TraderPlacers.Service to communicate with our Akka.CQRS.TradeProcessor.Service using a clustered consistent hashing router.

Rather than recap how clustered routers and, more specifically, how consistent hash routers work in this lesson, we're going to redirect you to a blog post we wrote on this subject: "Distributing State in Akka.Cluster Applications". **Please read that post before moving onto our exercise if you aren't familiar with clustered routers already**.

Now, let's enable Akka.Cluster inside both of these services and put them into communicate using a consistent hash router.

If you look inside `src/Akka.CQRS.TradePlacers.Service/Program.cs`, you'll see that we're already using Akka.Cluster's APIs in a couple of areas:

```
static int Main(string[] args)
{
    var config = File.ReadAllText("app.conf");
    var conf =
ConfigurationFactory.ParseString(config).BoostrapApplication(new
AppBootstrapConfig(false, true));

    var actorSystem = ActorSystem.Create("AkkaTrader", conf);

    var tradeRouter = actorSystem.ActorOf(
        Props.Empty.WithRouter(new ClusterRouterGroup(
            new ConsistentHashingGroup(new[]
{"/user/orderbooks"},

TradeEventConsistentHashMapping.TradeEventMapping),
            new ClusterRouterGroupSettings(10000, new []{
"/user/orderbooks" },
                                           true, useRole:"trade-
processor"))), "tradesRouter");

    Cluster.Cluster.Get(actorSystem).RegisterOnMemberUp(() =>
    {
```

```
        var shardRegionProxy = tradeRouter;
        var subManager = new
ActorTradeSubscriptionManager(tradeRouter);
        foreach (var stock in AvailableTickerSymbols.Symbols)
        {
            var max = (decimal)ThreadLocalRandom.Current.Next(20,
45);
            var min = (decimal)
ThreadLocalRandom.Current.Next(10, 15);
            var range = new PriceRange(min, 0.0m, max);

            // start bidders
            foreach (var i in Enumerable.Repeat(1,
ThreadLocalRandom.Current.Next(1, 6)))
            {
                actorSystem.ActorOf(Props.Create(() => new
BidderActor(subManager, stock, range, shardRegionProxy)));
            }

            // start askers
            foreach (var i in Enumerable.Repeat(1,
ThreadLocalRandom.Current.Next(1, 6)))
            {
                actorSystem.ActorOf(Props.Create(() => new
AskerActor(subManager, stock, range, shardRegionProxy)));
            }
        }
    });

    // start Petabridge.Cmd (for external monitoring /
supervision)
    var pbm = PetabridgeCmd.Get(actorSystem);
    pbm.RegisterCommandPalette(ClusterCommands.Instance);
    pbm.RegisterCommandPalette(ClusterShardingCommands.Instance);
    pbm.RegisterCommandPalette(RemoteCommands.Instance);
    pbm.Start();

    actorSystem.WhenTerminated.Wait();
```

```
        return 0;
    }
```

This code will fail to execute at the moment, because Akka.Cluster isn't loaded via HOCON. So let's fix that problem first by adding the following HOCON into `src/Akka.CQRS.TradePlacers.Service/app.conf` :

```
akka {
        actor {
                provider = cluster

                deployment{
                        /tradesRouter {
                                router = consistent-hashing-group
                                routees.paths = ["/user/orderbooks"]
                                cluster {
                                        enabled = on
                                        use-role = trade-
processor
                                }
                        }
                }
        }

        remote {
                dot-netty.tcp {
                        hostname = "127.0.0.1"
                        port = 0
                }
        }

        cluster {
                #will inject this node as a self-seed node at run-
time
                seed-nodes =
["akka.tcp://AkkaTrader@127.0.0.1:5055"]
                roles = ["trader", "trade-events"]
```

```
        }
    }
```

This HOCON will do the following things to the Akka.CQRS.TradePlacers.Service:

1. Enable Akka.Cluster via the `akka.actor.provider = cluster` setting;
2. Configures a clustered consistent hash router (`/user/tradesRouter`) to route messages using consistent hash routing to actors at the path `/user/orderbooks` on Akka.Cluster nodes that are running with a role type of `trade-processor` ;
3. Tells Akka.Remote to listen for incoming connections on 127.0.0.1:{random port} - this will also be the network address we use to identify ourselves inside the cluster;
4. Tells Akka.Cluster to attempt to join a seed node located at `akka.tcp://AkkaTrader@127.0.0.1:5055` at startup; and
5. Tells Akka.Cluster to label this node as having both “trader” and “trade-events” roles inside the cluster.

Bonus Exercise: why did we choose port 0 for our inbound port address for the Akka.CQRS.TradePlacers.Service, but not for the Akka.CQRS.TradeProcessor.Service?

You’ll notice that we already have some code designed to create this `/user/tradesRouter` router declared in C# inside `Program.cs` :

```
 var tradeRouter = actorSystem.ActorOf(
        Props.Empty.WithRouter(new ClusterRouterGroup(
            new ConsistentHashingGroup(new[]
{"/user/orderbooks"},

TradeEventConsistentHashMapping.TradeEventMapping),
            new ClusterRouterGroupSettings(10000, new []{
"/user/orderbooks" },
                                           true, useRole:"trade-
processor"))), "tradesRouter");
```

This was done so we could apply the `TradeEventConsistentHashMapping.TradeEventMapping` function to this router, which we

can't do via configuration:

```
/// <summary>
/// Creates a <see cref="ConsistentHashMapping"/>
/// </summary>
public static class TradeEventConsistentHashMapping
{
    public static readonly ConsistentHashMapping
TradeEventMapping = msg ⇒
    {
        if (msg is IWithStockId s)
        {
            return s.StockId;
        }

        return msg.ToString();
    };
}
```

This function is used to help extract the consistent hashing key that the router needs to compute which Akka.CQRS.TradeProcessor.Service instance we're going to route each individual message to - **all messages with the same hash key will all be routed to the same node**. Consistent hash routers need some way to determine how to find the hash key on each message, and a `ConsistentHashMapping` delegate is one option.

Bonus Exercise: what are some of the other ways we could route relevant trade events to the Akka.CQRS.TradeProcessor.Service aside from a consistent hashing router? Why, or why not, would those work?

The Akka.CQRS.TradePlacer.Service should be good to go. Now let's update the Akka.CQRS.TraceProcessor.Service.

Open the `src/Akka.CQRS.TradeProcessor.Service/app.conf` file and add the following HOCON inside the `akka` block:

```
actor {
        provider = cluster
```

```
    }

    remote {
            dot-netty.tcp {
                        hostname = "127.0.0.1"
                        port = 5055
                    }
    }

    cluster {
            #will inject this node as a self-seed node at run-time
            seed-nodes = ["akka.tcp://AkkaTrader@127.0.0.1:5055"]
            roles = ["trade-processor" , "trade-events"]
    }
```

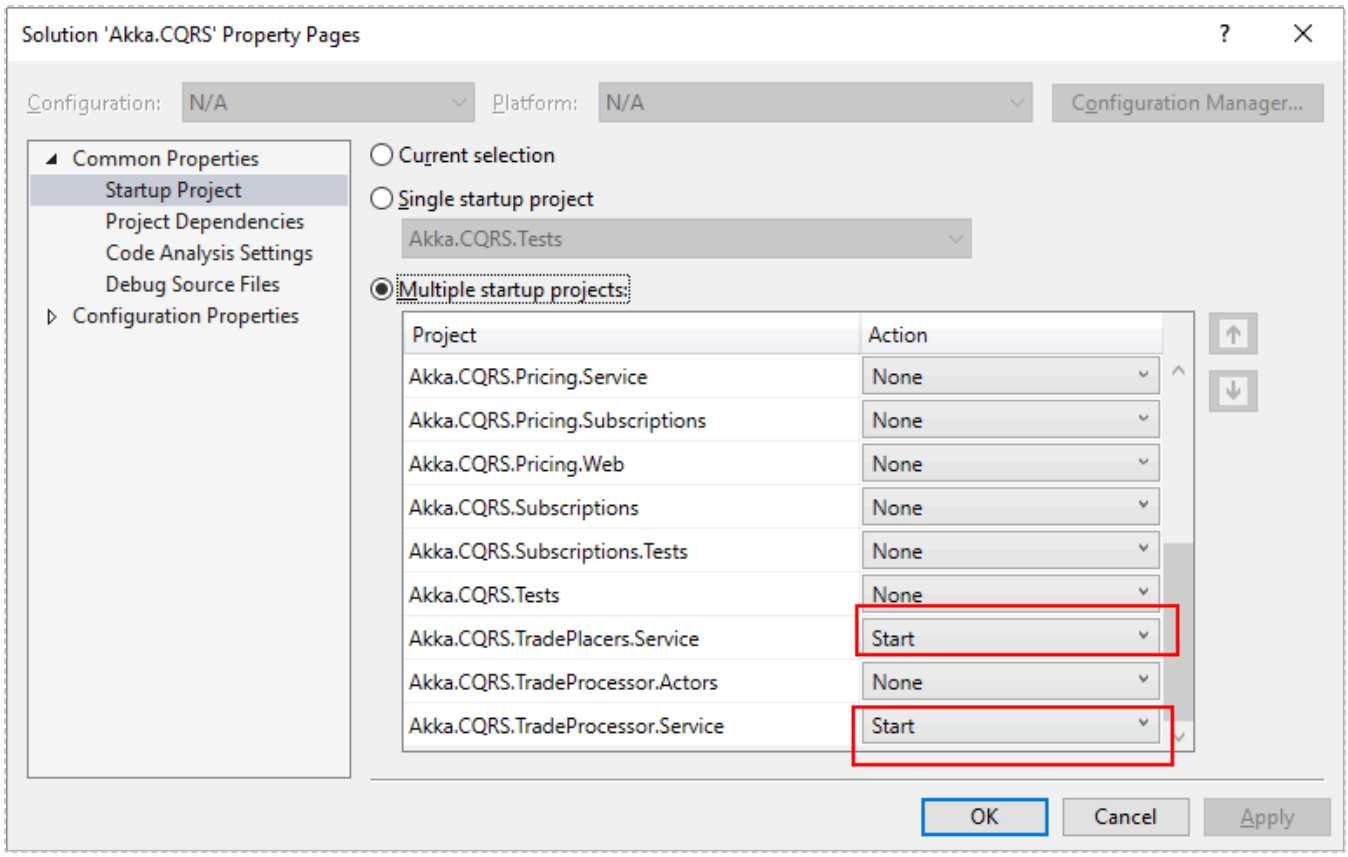
This HOCON will do the following things to the Akka.CQRS.TradeProcessor.Service:

1. Enable Akka.Cluster via the `akka.actor.provider = cluster` setting;
2. Tells Akka.Remote to listen for incoming connections on 127.0.0.1:5055 - this will also be the network address we use to identify ourselves inside the cluster;
3. Tells Akka.Cluster to attempt to join a seed node located at `akka.tcp://AkkaTrader@127.0.0.1:5055` at startup; and
4. Tells Akka.Cluster to label this node as having both “trade-processor” and “trade-events” roles inside the cluster.

You’ll notice that the Akka.CQRS.TradeProcessor.Service *uses itself* as a seed node in the cluster - that’s a standard practice. *Seed nodes need to know that they’re the seed nodes*. The cluster will become fully operation once the Akka.CQRS.TradePlacers.Service joins one of the Akka.CQRS.TradeProcessor.Service instances.

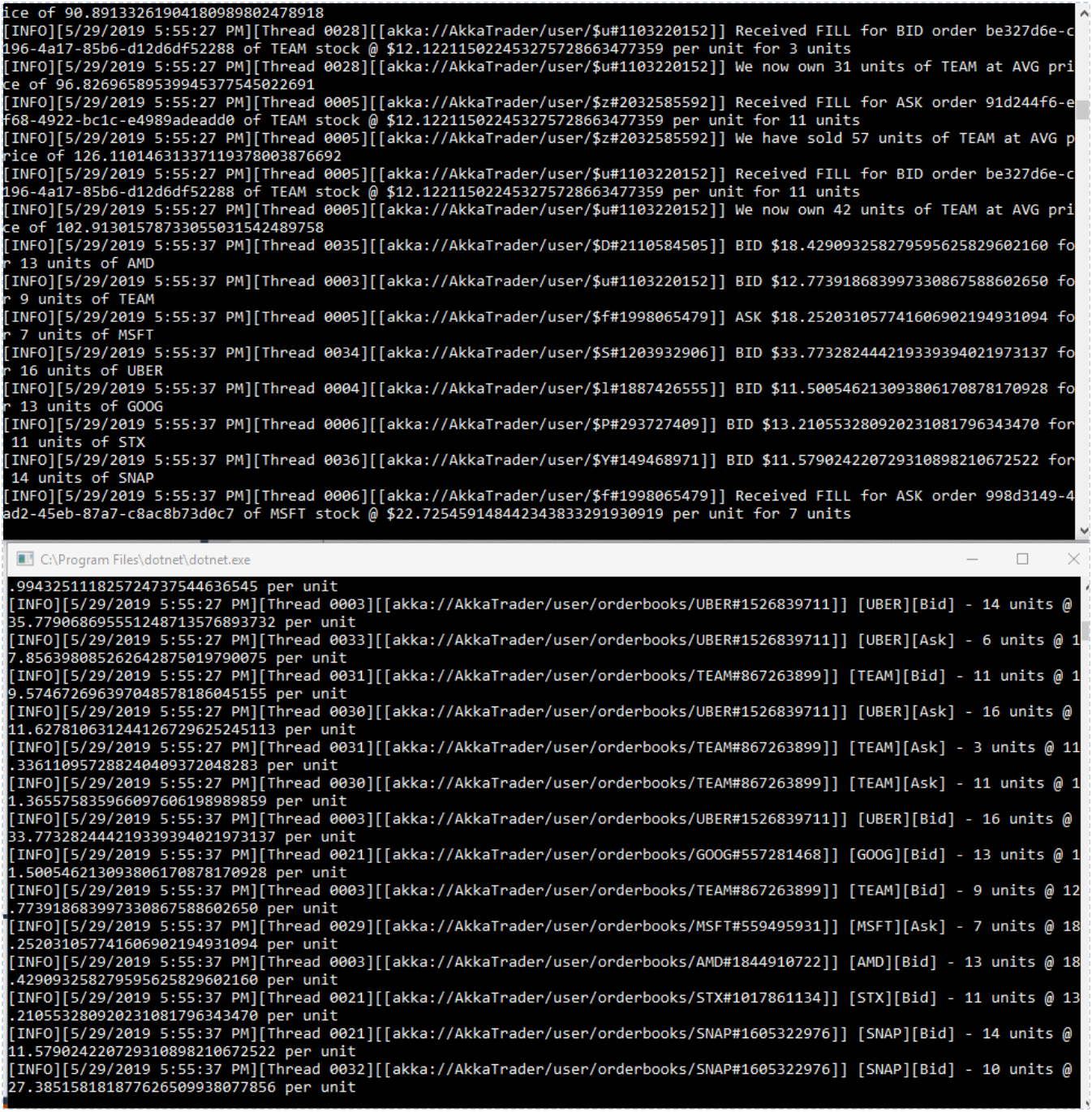
Running the Code

Now that everything is in-place, rebuild the solution, and then set the following startup projects inside the Akka.CQRS.sln:



To get access to this dialog, right click on the Akka.CQRS file in Solution Explorer and select **Select Startup Projects**.

Now run the solution. You should see output similar to the following:



This means that the Akka.CQRS.TradePlacers.Service is able to successfully send `Bid` and `Ask` events to the Akka.CQRS.TradeProcessor.Service and receive `Fill` events in return when a trade is matched.

Using Petabridge.Cmd to View the Cluster

As a final step for this section, we should learn how to manage and manipulate our cluster using Petabridge.Cmd - all of the nodes in the Akka.CQRS solution are equipped with the Petabridge.Cmd.Host so they can receive connections from the `pbm` client.

While your cluster is running, in a new terminal window try the following:

```
PS> pbm 127.0.0.1:9110 cluster show
```

And you should see the following output, assuming you haven't launched any additional Akka.CQRS.TradePlacers.Service nodes from Visual Studio:

```
akka.tcp://AkkaTrader@127.0.0.1:5054 | [trade-events,trader] | up
|
akka.tcp://AkkaTrader@127.0.0.1:5055 | [trade-events,trade-
processor] | up |
Count: 2 nodes
```

Excellent! Now try kicking one of the nodes out of the cluster using the `cluster leave` command:

```
PS> pbm 127.0.0.1:9110 cluster leave
```

This will force the current node to leave the cluster and, when it's all said and done, have its process exit.

```
Leaving cluster...
akka.tcp://AkkaTrader@127.0.0.1:5054 is now LEAVING.
akka.tcp://AkkaTrader@127.0.0.1:5054 is now EXITING.
akka.tcp://AkkaTrader@127.0.0.1:5054 is now REMOVED. Leave
complete.
```

Bonus Exercise: can you create a 3 or a 4-node cluster by launching additional Akka.CQRS.TradePlacers.Service instances from Visual Studio? What does the `cluster show` output look like then? What are some of the other Petabridge.Cmd Akka.Cluster commands we can use here?

Next

Now that we have a functioning Akka.NET cluster, it's time for us to take our work to the next level by packaging our Akka.NET applications into their own Docker containers: Lesson 2 - Docker-izing Akka.NET and Akka.Cluster.

// PETABRIDGE

PLATFORM

Akka.NET

Akka.Cluster

Akka.Streams

Akka.Persistence

Phobos

Petabridge.Cmd

SERVICES

Design-Time Support

Production Support

Architecture Reviews

Training

Consulting

RESOURCES

Bootcamp

Documentation

Blog

Newsletter

CONNECT

GitHub

Twitter / X

YouTube

About Us

Contact Us

© 2015-2026 Petabridge, LLC · Conroe, TX

// building distributed systems since 2015